# **RadeonNav:** Skill-Aware Quadruped Navigation with Natural Behaviours on the ROCm Platform


**Abstract**

Quadruped locomotion skills and local obstacle-avoidance planners are usually developed separately. A geometric planner does not know which obstacles the robot's current skills can cross, while the skill policy assumes that task labels and goals are supplied. This report connects a point-cloud local planner (NeuPAN) to a quadruped locomotion stack through the policy's existing observation channels, without modifying any network architecture. Single-obstacle tests determine which task classes the robot can traverse and how much they slow it down. The planner routes around obstacles that the robot cannot traverse, while sparse waypoints guide precise approaches to skills. During jumps, the target heading is held fixed; elsewhere, the planner handles steering and dynamic avoidance. Only the depth encoder is fine-tuned, using data collected while the planner controls the robot.

On a $28\,\mathrm{m}\times10\,\mathrm{m}$ serpentine course, the robot completes all task categories in a single continuous rollout. Across 12 procedural terrain realisations, 3 trials complete the course in $79.7$–$170.2\,\mathrm{s}$; the remaining 9 trials terminate in stalls, with 4 at the tire hoop, 3 at the bar-hurdle detour, and 2 in the second row. No trial results in a fall. Both depth encoder fine-tuning and end-to-end policy inference run on a single AMD Radeon GPU with the ROCm stack.

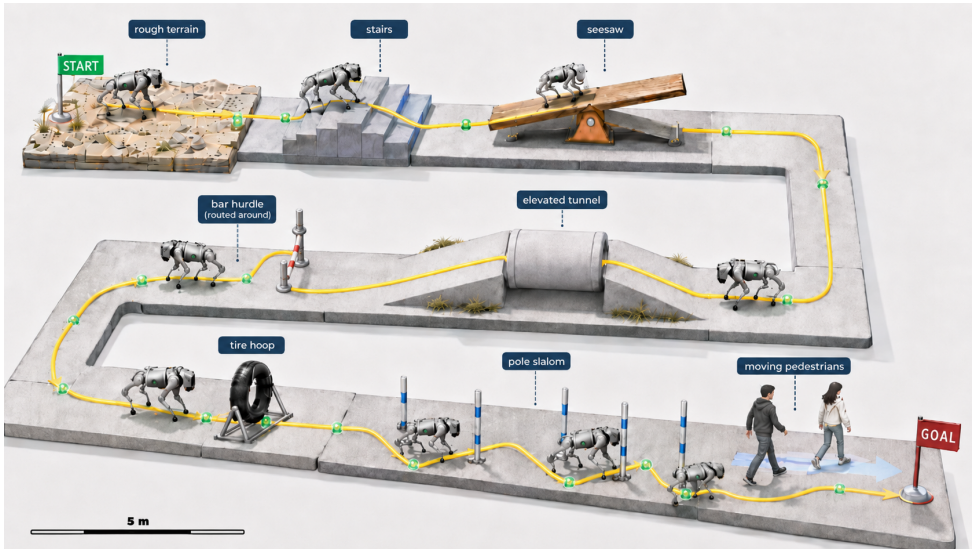




**Figure 1. Task overview.** A Unitree Go2 traverses a $28\,\mathrm{m}\times10\,\mathrm{m}$ serpentine course covering eight task categories—rough terrain, stairs, a seesaw, an elevated tunnel, a bar-hurdle detour, a tire-hoop jump, a pole slalom, and moving pedestrians—in one continuous rollout of roughly $80\,\mathrm{s}$. Green ground markers are sparse waypoints used to define the route; the yellow trace is the robot's actual trajectory.

## 1 Introduction

Quadruped navigation requires more than collision avoidance. In environments with stairs, low passages, hurdles, narrow openings, or moving people, the robot must decide which features to traverse and which to bypass. Recent locomotion policies provide natural gait patterns and agile skills such as jumping, crawling, and incline traversal [1], while point-cloud planners avoid static and dynamic obstacles in the plane [2]. Yet these components are designed separately: the planner does not know what the robot's current skills can traverse, and the locomotion policy does not choose routes.

The single-task evaluation of Fu et al. [1] illustrates this disconnect. Each episode is given a ground-truth task label and a sequence of hand-authored goals. The task label conditions the TSC's skill selection, while the goals determine

the approach heading. This setup does not extend directly to long routes: the planner may choose a geometrically valid path that the locomotion policy cannot traverse, and frequent heading updates can disrupt a jump approach.

We connect NeuPAN to the locomotion stack through the policy's existing inputs. We first test each task class with the current robot, policy, and simulator. We then use the measured success and traversal speed to build the route: regions the robot cannot cross are marked as obstacles for the planner, while slower terrain receives a higher path cost.

At runtime, the task class is determined from route progress, and headings come from either NeuPAN or nearby waypoints. Waypoints guide the approach to each task, and the target heading stays fixed during a jump; elsewhere, NeuPAN handles steering and dynamic avoidance. All signals use the existing high-level observation channels. Only the depth encoder is fine-tuned, using observations collected under planner control.

The report makes the following contributions:

- an end-to-end navigation system that uses single-obstacle tests to decide which regions to cross or bypass, selects the task class from route progress and the heading from the planner or waypoints, and adapts the depth encoder with data collected under planner control;
- new assets and a depth-sensor implementation in UniLab [3], together with a ROCm adaptation of NeuPAN's DUNE model;
- training and inference on one AMD Radeon GPU under ROCm, with CPU-side physics and planning during full-course evaluation.

> Readers primarily interested in training and platform measurements may proceed directly to Section 4.

# 2 Methodology

## 2.1 System overview

The system comprises three modules—depth perception, local planning, and the locomotion stack—connected as in Figure 2. The depth module encodes onboard images into a terrain representation. The planner generates local trajectories and heading commands from planar point clouds. The locomotion stack combines the high-level skill selector, which selects behaviours and produces motion commands, with the low-level gait controller, which controls the joints. The modules communicate only through the high-level observation vector, without changing any network.

Notation: $\mathbf{s}_t = (x_t, y_t, \theta_t)$ is the planar robot pose, $\mathbf{u}_t$ the base velocity command, $\mathbf{o}_t^{\mathrm{p}} \in \mathbb{R}^{65}$ the proprioceptive block of the high-level observation, $\mathbf{h}_t \in \mathbb{R}^{570}$ the proprioceptive history, $I_t$ the depth image, $\ell \in \{0, \ldots, 5\}$ the terrain class, and $\sigma$ the arc-length coordinate along the reference path.

The local planner runs at $10\,\mathrm{Hz}$ and converts its trajectory into yaw angles at two look-ahead points. Sparse route waypoints provide another pair of yaw angles. The planner headings are used in open space and around moving obstacles; waypoint headings are used when a task requires a precise approach. The selected headings and the current task class are written into the high-level observation. The high level selects a skill and produces motion commands, and the low level converts them into twelve joint targets at $50\,\mathrm{Hz}$. All networks run in half precision.

## 2.2 Depth perception

The perception module takes $58 \times 87$ depth images from the front-facing body camera, with mounting pose and field of view matching the upstream implementation. The encoder is a convolutional backbone with a hidden-size 512 GRU, producing a 32-D terrain latent, a 2-D yaw prediction, and a 6-D terrain-class prediction:

$$\left[\mathbf{e}_t, \hat{\boldsymbol{\psi}}_t, \hat{\mathbf{c}}_t\right] = f_{\mathrm{enc}}\left(I_t,\ \mathbf{o}_t^{\mathrm{p}};\ \mathbf{z}_{t-1}\right), \qquad \mathbf{e}_t \in \mathbb{R}^{32}, \quad \hat{\boldsymbol{\psi}}_t \in \mathbb{R}^2, \quad \hat{\mathbf{c}}_t \in \Delta^5, \tag{1}$$

where $I_t$ is the depth image, $\mathbf{o}_t^{\mathrm{p}}$ the proprioceptive observation, and $\mathbf{z}_{t-1}$ the GRU state.

The upstream policy predicts goal heading and terrain class from $\hat{\boldsymbol{\psi}}_t$ and $\hat{\mathbf{c}}_t$. This work replaces both outputs: the planner or route waypoints provide the heading, and route progress provides the task class. Only the terrain representation $\mathbf{e}_t$ is retained. It replaces the simulator-provided height scan used by the teacher, allowing the high-level network to operate on onboard depth without changing its architecture or weights.

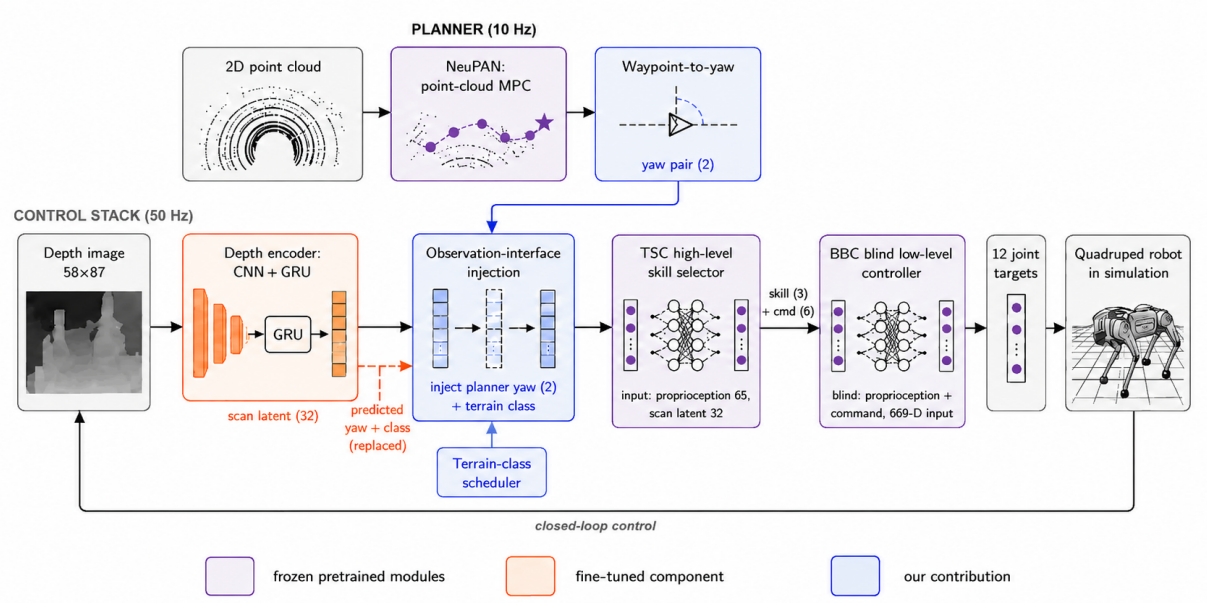



**Figure 2. System architecture.** The point-cloud MPC planner generates local trajectories in free space at 10 Hz and handles pedestrians, poles, and virtual walls that represent obstacles the robot cannot traverse. Near each task, route waypoints provide the precise approach required by dynamic skills. Planner heading, waypoint heading, and terrain class enter the skill selector through existing observation channels; the target heading is held constant during jumps. The gait controller uses only proprioception and high-level commands, emitting joint targets at 50 Hz. Only the depth encoder is fine-tuned; all other networks use the released weights. Neural networks run on a single AMD Radeon GPU; rigid-body simulation and the planner's optimisation run on the CPU.

## 2.3 Local planning and route construction

The planning module combines the upstream point-cloud model-predictive controller, NeuPAN, with code that builds its reference route and converts its output into policy inputs. NeuPAN solves local trajectories from a reference path and point cloud. Our code creates those inputs, adds regions that require a detour to the point cloud, and converts the planned trajectory into the two yaw angles consumed by the policy.

**Native planner (NeuPAN).** The planner's algorithmic formulation and released weights follow the upstream implementation unchanged. This work supplies its configuration and inputs; the execution-only ROCm adaptation of the DUNE forward pass is described in Section 4.2. The planner approximates the robot's planar footprint by a convex polygon:

$$\mathcal{B} = \left\{ \mathbf{p} \in \mathbb{R}^2 \mid G\mathbf{p} \leq \mathbf{h} \right\}. \tag{2}$$

A learned distance module [2] predicts, per obstacle point, the dual variables of the convex-set distance and yields a distance estimate differentiable in the robot state:

$$(\boldsymbol{\mu}_j, \boldsymbol{\lambda}_j) = f_{\mathrm{DUNE}}(\mathbf{p}_j), \qquad \boldsymbol{\lambda}_j = -RG^\top \boldsymbol{\mu}_j, \qquad d_j = \boldsymbol{\mu}_j^\top \left( G\mathbf{p}_j - \mathbf{h} \right). \tag{3}$$

The distance estimates enter a finite-horizon trajectory optimisation, whose objective we summarise as

$$\begin{aligned} \min_{\mathbf{s},\mathbf{u},\mathbf{d}} \quad & J_{\mathrm{track}} + J_{\mathrm{vel}} + J_{\mathrm{prox}} - \eta \sum_{t=1}^{T} d_t + J_{\mathrm{obs}} \\ \text{s.t.} \quad & \mathbf{s}_{t+1} = A_t \mathbf{s}_t + B_t \mathbf{u}_t + C_t, \\ & (\mathbf{s}_t, \mathbf{u}_t) \in \mathcal{S} \times \mathcal{U}, \qquad d_{\min} \leq d_t \leq d_{\max}, \end{aligned} \tag{4}$$

where $J_{\mathrm{track}}$, $J_{\mathrm{vel}}$, and $J_{\mathrm{prox}}$ are the reference-tracking, velocity, and proximal terms, $d_t$ is the safety clearance between trajectory and obstacles, and $J_{\mathrm{obs}}$ penalises the linearised obstacle constraints. The planner tracks the reference path in receding-horizon fashion and solves by unrolled iterations. This work uses horizon $T = 10$, control interval $0.1\,\mathrm{s}$, $K = 2$ iterations per cycle, and a footprint matching the Go2.

**Route construction and policy headings (this work).** The native planner requires a reference path and a point cloud but does not specify how to build them or how to feed its output to the locomotion policy. We provide these functions in three steps.

First, we construct the reference path. For single-task evaluation, the path is planned on a two-layer 2.5D map. One layer contains physical obstacles inflated by the $0.40\,\mathrm{m}$ robot radius; the traversable layer contains task regions passable by skill, kept uninflated to preserve the source tasks' true passage width. The map is searched eight-connected at $0.10\,\mathrm{m}$ resolution and the path resampled at $0.05\,\mathrm{m}$ arc length. For the full course, the reference path is resampled from sparse waypoints placed at task entries and exits, around the two free-space turns, and through the pole slalom. The planner determines the local trajectory between successive waypoints.

Second, we use the single-obstacle results in route planning. For class $k$, with traversability flag $\tau_k \in \{0, 1\}$, mean traversal speed $v_k$, and flat-ground speed $v_{\mathrm{free}}$, the terrain cost multiplier and path cost are

$$c_k = \max\left(\frac{v_{\mathrm{free}}}{v_k},\, 1\right), \qquad J(\Gamma) = \int_{\Gamma} c_{\ell(\sigma)}\,\mathrm{d}\sigma. \tag{5}$$

Traversable but slower terrain costs more. Classes faster than flat ground are clamped to 1, so these measurements never attract the route towards a particular terrain class. Regions the robot cannot traverse are excluded from finite-cost paths and added to the planner's point cloud as virtual obstacles:

$$\mathcal{P}_t = \mathcal{P}_t^{\mathrm{dyn}} \cup \mathcal{P}^{\mathrm{stat}} \cup \mathcal{P}^{\mathrm{virt}}, \qquad \mathcal{P}^{\mathrm{virt}} = \bigcup_{k:\,\tau_k = 0} \partial\mathcal{A}_k. \tag{6}$$

These virtual obstacles exist only in the planner; the physical scene, collision geometry, and depth observations are unchanged. In our tests, stairs and the hinged platform are traversable but slower than flat ground. The bar hurdle cannot be cleared reliably, so a virtual wall tells the planner to find a detour online with Equation (4).

Third, we generate the headings used by the policy. The policy consumes yaw angles rather than trajectories. We project the robot onto the route and track its progress by arc length. Progress is not allowed to move backwards, which avoids ambiguous projections on the serpentine parts of the route. Two points ahead of the robot are converted into clipped yaw angles as given in Equation (9). The current NeuPAN trajectory is converted in the same way, so the system can choose between planner and waypoint headings without changing the policy input format.

If the robot does not move far enough within a fixed time window, the current route segment is temporarily blocked and the planner searches for another path. This allows the robot to recover from moving obstacles or an incorrect single-obstacle result.

**Data flow.** The route and point cloud are passed to NeuPAN, which returns a local trajectory. The trajectory is converted into yaw angles and combined with the waypoint headings as described in Section 2.5. The planner updates at $10\,\mathrm{Hz}$; the heading choice and policy update run at $50\,\mathrm{Hz}$. Between planner updates, the waypoint headings continue to advance with the robot's progress along the route.

## 2.4 Locomotion policy

The locomotion stack is two-level. The high-level TSC skill selector selects a behaviour mode and generates continuous motion commands; the low-level BBC gait controller turns them into joint position targets.

The TSC observation comprises a 65-D proprioceptive block, the 32-D terrain latent, a 4-D explicit state estimate, and the 570-D proprioceptive history; the 65-D block itself contains 57 proprioceptive states, the 2-D heading input, and the 6-D terrain class. The network is a 512–256–128 MLP whose discrete head chooses among trot, canter, and jump, and whose continuous head emits velocity, body-height, and jump-height commands.

The BBC receives the 9-D conditioned command from the high level plus proprioception, state estimate, history latent, and proprioceptive history—669 inputs in total—and outputs twelve joint position targets at $50\,\mathrm{Hz}$. The low level uses no exteroception.

The two levels are

$$\left(z_t^{\mathrm{d}}, \mathbf{z}_t^{\mathrm{c}}\right) = \pi_{\mathrm{TSC}}\left(\mathbf{o}_t^{\mathrm{p}}, \mathbf{e}_t, \hat{\mathbf{x}}_t, \mathbf{h}_t\right), \qquad \mathbf{a}_t = \pi_{\mathrm{BBC}}\left(\mathbf{o}_t^{\mathrm{p},57}, \hat{\mathbf{x}}_t, \hat{\mathbf{g}}_t, \mathbf{c}_t, \mathbf{h}_t\right), \tag{7}$$

where $z_t^{\mathrm{d}}$ is the discrete behaviour mode, $\mathbf{z}_t^{\mathrm{c}} \in \mathbb{R}^6$ the continuous command, $\mathbf{c}_t$ the 9-D low-level conditioning formed by concatenating the continuous command with the mode one-hot, and $\mathbf{a}_t \in \mathbb{R}^{12}$ the joint targets.

Both the TSC and BBC modules, their training pipelines, and their released weights originate from Fu et al. [1]. In particular, the upstream BBC was trained on retargeted canine motion capture with an adversarial imitation objective. The natural gait patterns and body postures observed in this system are therefore inherited from those released weights rather than learned in this work. We use the released TSC, BBC, and state-estimator weights as-is. Our system changes only the heading and task-class channels of the high-level observation; it does not alter the joint-level actions. This preserves the upstream behaviours and skill repertoire.

The planner can therefore choose routes that use the available skills, but it cannot create motions that the locomotion policy never learned.

### 2.5 Policy inputs

The three modules connect through existing channels of the high-level observation. The heading from the planner or route waypoints is written into the 2-D yaw channel, and the task class selected from route progress is written into the 6-D one-hot channel:

$$\mathbf{o}_t^{\mathrm{p}}[57{:}59] \leftarrow \Delta\boldsymbol{\psi}_t, \qquad \mathbf{o}_t^{\mathrm{p}}[59{:}65] \leftarrow \mathrm{onehot}\big(\ell(\sigma_t)\big). \tag{8}$$

These eight channels replace the yaw and class predictions of Equation (1); every other observation is unchanged. The planner and policy share no network or hidden state.

**Choosing the heading source.** In open space, the planner must steer continuously around obstacles. A hoop jump instead needs a stable and precise approach. We therefore use waypoint headings near each task and planner headings elsewhere.

Near a task, reference points at $L_1 = 0.35\,\mathrm{m}$ and $L_2 = 0.70\,\mathrm{m}$ ahead are converted into two clipped yaw angles:

$$\psi_{i,t}^{\mathrm{wp}} = \mathrm{clip}\left(\mathrm{wrap}\left(\angle\left[\Gamma(\sigma_t + L_i) - \mathbf{s}_t^{xy}\right] - \theta_t\right),\ -\psi_{\max},\ \psi_{\max}\right), \qquad i \in \{1, 2\}. \tag{9}$$

Away from tasks, the heading comes from the planner trajectory. When a moving obstacle comes within $d_{\mathrm{safe}} = 2\,\mathrm{m}$, the planner also takes over near a task and reduces forward speed. The heading is selected as

$$\boldsymbol{\psi}_t = \begin{cases} \boldsymbol{\psi}_t^{\mathrm{pl}}, & \sigma_t \notin \mathcal{C} \text{ or } d_t^{\mathrm{obs}} < d_{\mathrm{safe}}, \\ \boldsymbol{\psi}_t^{\mathrm{wp}}, & \text{otherwise}, \end{cases} \tag{10}$$

where $\mathcal{C}$ contains the approach regions along the route. Right-angle turns, the hurdle detour, and pedestrian avoidance are handled by the planner, while waypoint headings provide the accuracy needed near tasks.

**Choosing the task class and holding the jump heading fixed.** Each task has a start and end position along the route, including margins for entering and leaving it. As the robot advances, the corresponding task class is written into the 6-D one-hot input.

For a jump, the absolute target heading is recorded when the robot enters the task region:

$$\bar{\psi} = \theta_{t_0} + \psi_{1,t_0}^{\mathrm{wp}}, \qquad \Delta\psi_t = \mathrm{wrap}\left(\bar{\psi} - \theta_t\right), \tag{11}$$

where $t_0$ is the entry step. $\bar{\psi}$ is held during execution, so the yaw input changes only when the robot itself rotates, not when the planner updates. If a moving obstacle enters the safety range, the target heading may be reset once. Normal heading selection resumes after the robot leaves the task region. Holding the heading fixed prevents small planner corrections from shifting the take-off direction or timing.

## 3 Environment and Data Collection

This work uses the locomotion policies released by Fu et al. [1], whose task environments, sensor paths, and evaluation were built on Isaac Gym. Since this project runs on UniLab [3], the tasks, sensor models, and policy interfaces had to be rebuilt on a different physics engine and rendering pipeline.

**Table 1. Fine-tuning dataset composition.** All three groups contain single-task or obstacle-free scenes of a few hundred control steps; the full course contributes no data. Episode counts are the final training set after merging two DAgger rounds.

| Data group | Collection scene | Episodes | Purpose |
|---|---|---|---|
| Original task setup | single obstacle, source setup | 125 | retain original task behaviour |
| Planner-controlled approach | single obstacle, planner controls approach | 75 | cover deployment approaches |
| Obstacle-free walking | obstacle-free, curvature/yaw sweep | 78 | retain ordinary walking |
| Total | — | 278 | 133 440 valid frames |

### 3.1 Task-scene and sensor reconstruction

**Task scenes.** Every obstacle class in the upstream task set is rebuilt as an independent scene fragment. Visual meshes are reused; collision geometry is converted from the visual meshes into primitive compounds, because the convex decomposition used by the source has no direct counterpart in the current contact solver.

For articulated obstacles the source dynamics and constraints are preserved where possible: the bar and the suspended hoop use prismatic joints with stiff springs and damping, and the seesaw a passive hinge with a rest angle. Perception is rebuilt for both inputs used by the policy: an onboard depth camera configured to the source mounting pose and field of view, and the simulator-provided height sampler used by the teacher. Its sampling locations and coordinates match the source implementation so that teacher supervision remains comparable.

A ledger records implementation differences between the two engines. An incorrect axis mapping for the seesaw inertia destabilised the simulation; correcting the mapping and adding ground stops restored stability.

We attribute the remaining gap partly to differences in the numerical realisation of contact manifolds and joint constraints. Prismatic joints and instantaneous contact impulses are not equivalent across the two engines; the resulting lower take-off impulse limits full-height bar clearance in the current environment.

**Depth sensor.** UniLab does not itself provide a depth camera, so one is reconstructed through the simulator's rendering path.

### 3.2 Dataset Preparation

Fine-tuning data are collected in closed-loop simulation, without manual trajectories or full-course runs. The dataset contains three types of scenes (Table 1). The first reproduces the upstream single-obstacle setup, retaining the original task behaviour. In the second, the planner controls the approach to the same obstacles, exposing the encoder to the trajectories and camera views seen at deployment. The third contains obstacle-free ground with path curvatures $\kappa \in [0, 1]$ and initial yaw offsets $\pm 0.4$, $\pm 0.8$, and $\pm 1.2$ rad, retaining the ordinary walking behaviour used over most of the course.

The student executes the trajectory while the simulator-based teacher labels each step. Training follows DAgger [4] over two rounds: 159 episodes first, then 119 more collected under the updated student, so the data contain the states the current policy actually visits, including its failures—278 episodes and 133 440 valid frames in total. Each episode contributes 480 frames and training runs 6 epochs.

## 4 Training

This section covers the training configuration, the runtime hardware split, and the ROCm adaptation.

### 4.1 Hardware

All neural computation runs on a single AMD Radeon GPU (gfx1100): the three policy networks infer in half precision, and the depth encoder is fine-tuned on the same GPU in single precision. Everything else runs on the CPU—rigid-body dynamics and contact solving, the planner's optimisation (including, in the long-course baseline, its learned distance field; a GPU hybrid is described in Section 4.2), and route search using the measured traversal costs. Depth images come from a headless rendering path as $58 \times 87$ frames; control runs at 50 Hz with two physics substeps per control step.

On this platform, three-network per-step inference takes $4.938\,\mathrm{ms}$ (30 runs of 500 steps, timed after depth capture, device-synchronised); the full planner call has a p95 of $24.435\,\mathrm{ms}$ at a $10\,\mathrm{Hz}$ replan rate; and four-worker data collection aggregates $67.64\,\mathrm{steps/s}$, bottlenecked by CPU-side rigid-body and contact solving—a single worker averages about $45.7\,\mathrm{ms}$ per step, of which GPU inference is roughly one tenth.

### 4.2 ROCm adaptation

The target platform is a single AMD Radeon GPU (gfx1100) in a dual-socket AMD EPYC 9334 host, running Ubuntu 24.04.4 with ROCm 7.2.1 and PyTorch 2.11.0+rocm7.2 (HIP runtime 7.2.26015); physics runs on the CPU under MuJoCo 3.8, and the planner's convex layer solves through cvxpylayers/ECOS. The container image was not archived. The deployed pipeline contains no custom GPU kernels: half-precision operators use the HIP-compatible interface, and the `cuda` device name on this platform refers to the AMD GPU. The two relevant implementation details are described below.

**UniLab.** The simulation framework is ROCm-native: physics runs on the CPU independently of any GPU backend; policy-side tensor computation executes through the HIP backend with no changes to model source or checkpoint loading; depth images come from headless EGL rendering (with a software fallback) and need no display server. The framework's optional CUDA helper path is not loaded by this baseline.

**NeuPAN's DUNE.** The planner's only learned module—the DUNE distance encoder—runs on ROCm. Obstacle points for all steps of the prediction horizon are batched into one single-precision forward pass on the AMD GPU, and the outputs are copied back to the CPU. The online convex optimisation remains on the CPU. On fixed inputs, the maximum absolute GPU–CPU error is $1.5 \times 10^{-6}$ and planner actions agree within $10^{-3}$; the DUNE forward drops from $5.05\,\mathrm{ms}$ to $0.894\,\mathrm{ms}$ ($5.65\times$), while the full planner call speeds up only slightly because the convex solve and data movement dominate.

### 4.3 Fine-tuning

Early experiments also trained the high-level skill selector with the new waypoint input. This reduced some errors in the training environment but performed worse in fixed tests under the deployment setup. Retraining the high level appears to change when skills are selected and to increase the difference between teacher inputs and deployment inputs. We therefore fine-tune only the depth encoder and keep the released weights for the two policy levels.

The three data groups and the iterative collection are described in Section 3. Model selection does not rely on training loss alone: every saved epoch is tested on a fixed set of tasks in the deployment setup, including stairs, hoop jumping, and ordinary walking. We select the checkpoint that passes the largest number of predefined checks.

Fine-tuning on freshly collected data on the target platform—278 episodes, 133 440 valid frames, 6 epochs, 120 updates, single precision—takes $89.717\,\mathrm{s}$ of wall clock, excluding collection and the per-epoch checkpoint tests; the same-size run on historical data takes $83.818\,\mathrm{s}$ on the same GPU. End-to-end time is therefore spent almost entirely on data collection and model selection rather than gradient updates, consistent with the CPU-physics bottleneck identified above.

## 5 Evaluation

### 5.1 Full-Course Setup and Criteria

The task route spans approximately $28\,\mathrm{m}\times10\,\mathrm{m}$ within a scene footprint of about $29\,\mathrm{m}\times11\,\mathrm{m}$. Its three switchback rows contain eight task categories (Figure 3)—rough terrain, stairs, a seesaw, a ramp-and-elevated-tunnel combination, the bar hurdle, the tire-hoop jump, a four-pole slalom, and moving pedestrians in the goal area. The layout also inserts two stretches of free-space navigation without a scheduled terrain skill between the task segments so that the planner's contribution can be observed in isolation. Obstacle assets are rebuilt from the upstream task set; the rough terrain is a procedural height field.

The reference route is defined by sparse waypoints at task entries and exits, around the two free-space turns, and through the pole slalom. The planner determines the local trajectory between successive waypoints. Because the robot could not reliably clear the bar hurdle in the single-obstacle test, a virtual wall tells the planner to detour around it. The detour in the figure is planned online rather than scripted.

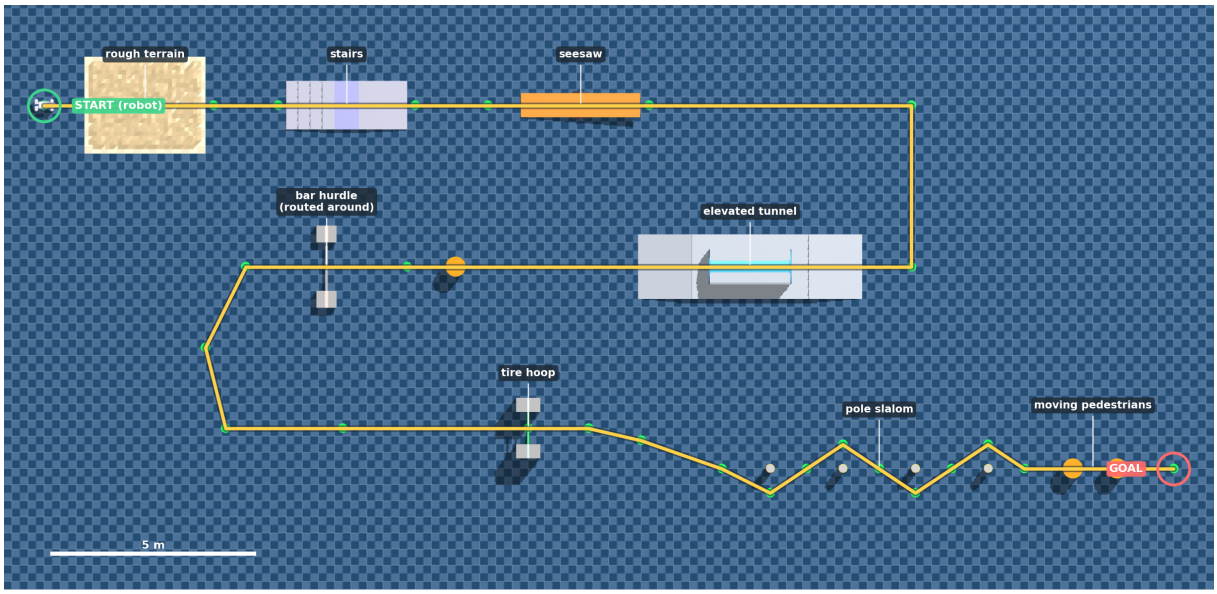



**Figure 3. Course top view (MuJoCo).** Three switchback rows chain eight task categories and two free-space turns. The yellow polyline is the reference route, green dots are its sparse waypoints, and orange cylinders are the moving pedestrians. The planner computes the bar-hurdle detour online because the robot could not reliably clear the bar in the single-obstacle test.

All results are computed from run records rather than demo footage. The evaluator records arrival times for each segment, the side on which each slalom pole is passed, collisions, falls, and planner latency.

## 5.2 Full-course results

Per-segment results of the reference run are in Table 2; the raw run records are under `docs/results/`. The robot completes the ten route segments in one continuous rollout: $79.68\,\mathrm{s}$, path length $74.05\,\mathrm{m}$, mean speed $0.93\,\mathrm{m\,s^{-1}}$; no falls, zero pedestrian collisions, and 28 control steps with contacts marked as forbidden by the evaluator ($0.70\%$), planner p95 $24.435\,\mathrm{ms}$. The two free-space stretches total $13.41\,\mathrm{m}$, $18\%$ of the path.

Among segment speeds, only the hurdle detour is markedly slow and long ($0.78\,\mathrm{m\,s^{-1}}$, $10.33\,\mathrm{m}$) because it adds distance and turning. Every other segment runs close to the flat-ground reference speed.

**Table 2. Reference run, per segment.** Arrival step is the control step at which the segment's positional criterion holds; path length is a polyline sum over the archived trajectory and slightly underestimates true arc length. Turns 1 and 2 use planner steering without a scheduled terrain skill. No falls, zero pedestrian collisions.

| Segment | Arrival step | Time (s) | Path (m) | Speed ($\mathrm{m\,s^{-1}}$) |
|---|---|---|---|---|
| Rough terrain | 206 | 4.12 | 4.14 | 1.00 |
| Stairs | 445 | 4.78 | 4.64 | 0.97 |
| Seesaw | 765 | 6.40 | 6.37 | 1.00 |
| Turn 1 (free space) | 1228 | 9.26 | 9.75 | 1.05 |
| Ramp and elevated tunnel | 1737 | 10.18 | 8.68 | 0.85 |
| Bar-hurdle detour | 2401 | 13.28 | 10.33 | 0.78 |
| Turn 2 (free space) | 2575 | 3.48 | 3.66 | 1.05 |
| Tire-hoop jump | 3053 | 9.56 | 10.09 | 1.06 |
| Pole slalom | 3647 | 11.88 | 11.40 | 0.96 |
| Pedestrians and goal | 3984 | 6.74 | 4.99 | 0.74 |
| Total | 3984 | 79.68 | 74.05 | 0.93 |

The hoop segment is sensitive to when the jump class begins. Starting it $0.8\,\mathrm{m}$ earlier along the route reduced the pause before the hoop from about $6\,\mathrm{s}$ to $0.4\,\mathrm{s}$. The evaluation records "passed through the ring" as its own check, so a sideways bypass cannot be mistaken for a pass by the along-path arrival marker.

### Success rate and time distribution

The rough-terrain height field is the only course element that varies with the seed. Across 12 seeds with all other conditions fixed, 3 trials complete the course in 79.7–170.2 s (Table 3). The reference run is the fastest successful trial rather than a typical one. The remaining 9 trials terminate in stalls: 4 at the hoop, 3 at the hurdle detour, and 2 in the second row. No trial results in a fall.

**Table 3. Full course over 12 terrain realisations.** Only the rough-terrain height field varies with the seed; geometry, weights, control-side constants, and start pose are identical. "Checks" counts the nine intermediate route-progress criteria before the final goal (the criteria are independent, not sequential gates); "ring offset" is the lateral offset at the hoop plane, and more than 0.40 m means the ring was not passed. No run falls.

| Seed | Verdict | Arrival step | Time (s) | Checks | Ring offset (m) | Stopped at |
|---|---|---|---|---|---|---|
| 7 | pass | 3984 | 79.68 | 9 | 0.047 | goal |
| 17 | pass | 4828 | 96.56 | 9 | 0.001 | goal |
| 37 | pass | 8511 | 170.22 | 9 | 0.029 | goal |
| 11 | fail | — | — | 7 | 0.005 | hoop |
| 31 | fail | — | — | 7 | 0.062 | hoop |
| 41 | fail | — | — | 7 | 0.000 | hoop |
| 53 | fail | — | — | 7 | 0.111 | hoop |
| 13 | fail | — | — | 5 | 4.137 | hurdle wall |
| 23 | fail | — | — | 5 | 4.075 | hurdle wall |
| 47 | fail | — | — | 5 | 4.080 | hurdle wall |
| 29 | fail | — | — | 4 | 8.033 | mid second row |
| 43 | fail | — | — | 3 | 8.050 | mid second row |

## 5.3 Comparison

To compare the effects of different high-level inputs, four configurations are evaluated on corresponding task fixtures (Table 4). All four share the same released gait controller; they differ only in the high-level inputs. The teacher's heading comes from hand-authored goal sequences, and its task class and height scan come directly from the simulator. The student infers both heading and class from depth. Planner-only and ours receive only a 2-D route. The teacher and student follow the upstream single-task setup (10 s, 500 steps, flat ground plus one obstacle), while planner-only and ours run on 8–12 m corridors. Only pass/fail outcomes, not arrival steps, are therefore compared across configurations.

Representative frames on six fixtures are shown in Figure 4. The student reaches none of the goals; the hurdle row shows it wedged under the bar. At the evaluation geometry, the teacher passes the other five fixtures but fails the full-height hurdle. Its hurdle frame uses the lowered bar only to illustrate the upstream jump behaviour. Planner-only detours every fixture. Our system crosses fixtures that passed the single-obstacle tests and detours around the rest.

**Table 4. Four-way comparison: configurations and outcomes.** The rows differ in two high-level input sources: approach heading and terrain class. The teacher obtains heading from hand-authored goals and task class from simulator labels; the student predicts both from depth. "Other terrain" covers rough terrain, stairs, seesaw, tunnel, and poles, none of which triggers a jump skill. Lateral offset distinguishes "pass" from "detour". The hurdle and hoop columns use 12 m corridors to rule out run-up length as a factor.

| Method | Heading source | Class source | bar | tire | pedestrians | other terrain |
|---|---|---|---|---|---|---|
| NeuPAN | sparse route waypoints | free space | detour | detour | avoid | all detour |
| TSC+BBC teacher | dense hand waypoints | simulator labels | fail[†] | pass | — | all pass |
| TSC+BBC student | dense hand waypoints | depth inference | fail | fail | — | all fail |
| Ours | sparse route waypoints | depth + route progress | detour | pass | avoid | all pass |

[†] Fails at the source training height of 0.20 m; passes when lowered to 0.10 m.

Planner-only reaches the goal on all eight segments but detours around every fixture (lateral offset 1.59–2.45 m), achieving reachability exclusively through avoidance while leaving the robot's motor skills unused. The upstream student fails all eight with onboard inputs. The released encoder exhibits the same failure pattern, suggesting that these failures are not introduced solely by fine-tuning. Driven through this work's interface, the same TSC and BBC weights reach all eight goals and match the teacher's six successful outcomes on the shared fixtures (the pole line by a deliberate detour where the teacher weaves the 1 m gaps). The system also handles the full-height hurdle, which

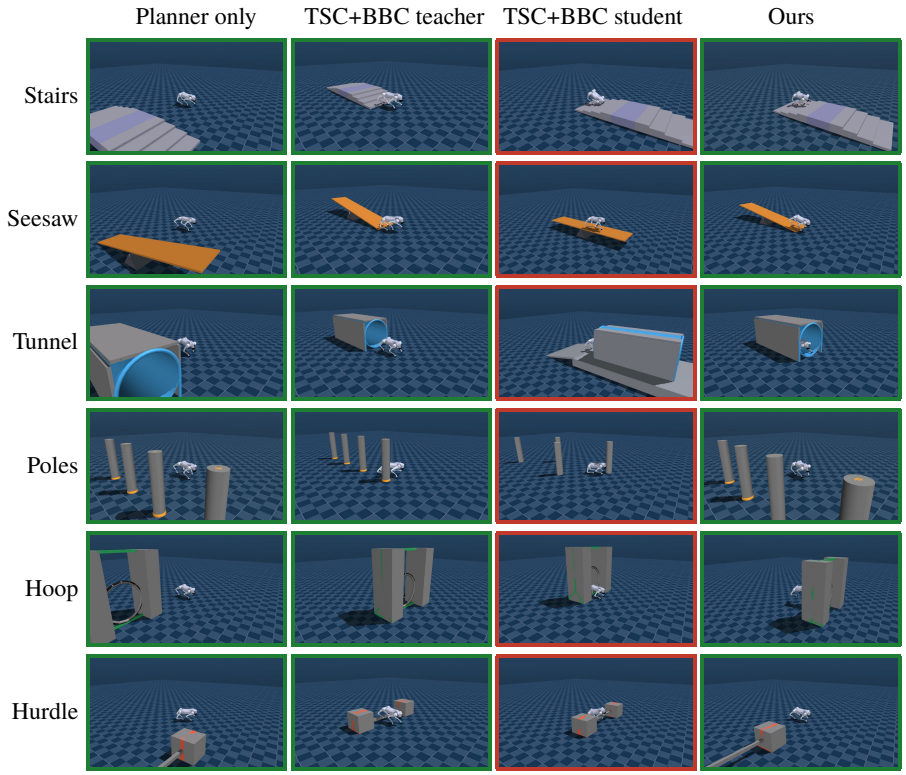



**Figure 4. Per-fixture frames for the four configurations.** Columns are the configurations of Table 4, rows are fixtures; green frames reached the goal, red frames did not. Camera azimuth is identical across columns; columns 1 and 4 come from 8–12 m corridor runs, columns 2 and 3 from the upstream single-task scenes. Hurdle row: the teacher frame is its lowered-bar pass at 0.10 m (it fails at the full 0.20 m, see the note of Table 4); the student wedges under the full-height bar; planner-only and ours both detour—the former treats every fixture as an obstacle, while the latter detours because the robot could not clear the bar in the single-obstacle test. Hoop row: ours is the moment of exiting the ring mid-jump; planner-only detours. The ours/stairs frame is taken from archived footage; all other frames are hand-picked representative moments.

the teacher fails, by detouring and completes the pedestrian segment absent from the teacher setup. The hurdle illustrates the route decision: because the current policy cannot reliably clear the bar, the planner detours instead of forcing the robot into it.

## 5.4 Ablation Study

The ablation keeps the waypoint headings but removes the measured traversal rules and the task classes derived from route progress. Waypoint headings alone recover four walking fixtures (rough terrain, stairs, seesaw, tunnel), indicating that heading prediction is the main source of the student's failures there: its mean absolute heading error is at least 0.72 rad on every fixture, whereas class prediction remains usable on most fixtures. Two fixtures need the traversal rules themselves. At the hurdle the encoder detects the bar and the policy initiates a jump, but, like the teacher given the simulator's task class, stops at the full-height obstacle; only the full system marks the bar as an obstacle and takes a detour. At the pole line the deployed form cannot weave the 1 m gaps the privileged teacher threads—the same single-obstacle test that marks the class untraversable—so the full system routes around the fixture, while the ablation, holding the straight corridor without traversal rules, grinds into the line for 939 forbidden-contact steps and never arrives. In the ablation, moving pedestrians also do not trigger planner control, so the robot walks straight ahead and collides. Relative to this ablation, selecting task classes from route progress and holding the jump heading fixed reduces hoop jump-skill steps from 203 to 66 and forbidden contacts from 19 to 3; this comparison measures their combined effect.

Table 5 gives the quantitative comparison. The arrival criterion matches the upstream waypoint success rate (0.4 m held for one control step), but upstream statistics average thousands of randomised episodes while ours count single deterministic runs per fixture, so the numbers are not directly comparable; speeds are unnormalised goal-directed projections. Against planner-only at the same arrival rate, this system raises traversable-terrain speed from 0.601 $\mathrm{m\,s^{-1}}$ to 0.970 $\mathrm{m\,s^{-1}}$; the ablation reaches 0.918 $\mathrm{m\,s^{-1}}$ on the same terrain but passes only $5/8$ overall, with

a forbidden-contact rate more than two orders of magnitude higher.

**Table 5. Quantitative comparison on the shared corridor.** Goal-directed speed is the body velocity projected onto the goal direction, computed by differencing the archived trajectories and averaged up to arrival. "Traversable 5" excludes the hurdle and the pole line—both classes the single-obstacle tests mark untraversable and this system deliberately detours—and the pedestrians, whose quality is determined by collisions rather than speed. The forbidden-contact rate is the fraction of all executed control steps with forbidden contacts.

| Configuration | Success | Goal-directed speed ($\mathrm{m\,s^{-1}}$) | | Forbidden-contact rate | Jump-skill steps |
|---|---|---|---|---|---|
| | (8 segs) | all 8 | traversable 5 | | |
| Planner only | $8/8$ | 0.597 | 0.601 | 0.0000 | 0 |
| Student + waypoints (ablation) | $5/8$ | 0.749 | 0.918 | 0.2179 | 261 |
| Ours | $8/8$ | **0.831** | **0.970** | 0.0008 | 98 |

# 6 Limitations and Future Work

The system has three main limitations. First, it cannot reliably clear the bar hurdle in the migrated simulation. We attribute this partly to contact and joint-constraint differences between the two physics engines, which reduce the take-off impulse, and partly to incorrect task-class predictions from some approach views. Second, the planner can only select among the skills already available; it cannot compensate for a motion that the locomotion policy has not learned. Third, evaluation remains in simulation, so sensor noise, actuator latency, and ground materials have not been tested on a real robot. We also do not yet report output differences across devices for identical inputs, per-module runtimes, or multi-seed confidence intervals for the per-class traversal measurements. These require new measurements rather than estimates.

Beyond the reported system, we ran a preliminary mixture-of-experts study with latent actions. It distils the released policies into separately trainable experts for different skills. This extension was not included in the final system: its jump pass rate is sensitive to take-off timing (about one in three), and although per-skill residual experts outperform the teacher in the training environment (the $0.125\,\mathrm{m}$ bar success rate rises from 0% to 82.8%), transfer to the deployment configuration still degraded performance after three consecutive rounds. The available experiments did not close this transfer gap.

Future work will first deploy the system on a physical Unitree Go2 to evaluate sim-to-real performance. We will then train per-skill experts specifically to reduce the gap between training and deployment and make the bar hurdle as reproducible as the other segments. Finally, we will extend the planar distance field to a 3-D representation with height and traversable surfaces for multi-level and volumetric route planning.

# References


[1] Huiqiao Fu, Haoyu Dong, Wentao Xu, Zhehao Zhou, Guizhou Deng, Kaiqiang Tang, Daoyi Dong, and Chunlin Chen. Learning diverse natural behaviors for enhancing the agility of quadrupedal robots. *Nature Communications*, 2026.

[2] Ruihua Han, Shuai Wang, Shuaijun Wang, Zeqing Zhang, Jianjun Chen, Shijie Lin, Chengyang Li, Chengzhong Xu, Yonina C Eldar, Qi Hao, et al. Neupan: Direct point robot navigation with end-to-end model-based learning. *IEEE transactions on robotics*, 41:2804–2824, 2025.

[3] Yufei Jia, Zhanxiang Cao, Mingrui Yu, Heng Zhang, Shenyu Chen, Dixuan Jiang, Meng Li, Xiaofan Li, Yiyang Liu, Junzhe Wu, et al. Unilab: A heterogeneous architecture for robot rl beyond gpu-dominant paradigms. *arXiv preprint arXiv:2605.30313*, 2026.

[4] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.